Introduction (2 min.)

- Décrire rapidement schéma et exemple (cheflieu Finistère -> Quimper !)
- Rappel graphe cyclique/acyclique. Représentation sous forme matricielle/liste d'adjacence
- Hypergraphe -> généralisation théorie des graphes non orientés. Description des arêtes et formalisation. Représentation comme liste d'adjacence
- Implémentation python (types). Bitfield car facile à manipuler, entiers non bornés en python et assez peu de sommet pour le cas d'usage.

Aspect théorique (4 min.)

- Pour introduire les outils utilisés après.
- Arité : nombre d'attributs. n-uplet : valeur possible d'un élément de la relation. Instance : ensemble (fini) de n-uplets de plusieurs relations.
- Opérations : projection (on ne garde que certains attributs), sélection (on ne garde que les n-uplets satisfaisant des conditions)
- Requête conjonctive : en-tête et corps de la requête. Clauses atomiques. Vecteur dans la base de données.
- Hypergraphe d'une requête : les variables communes font le lien entre les différentes relations. Intrinsèquement un hypergraphe.
- Arbre de jointure sous réserve d'existence. Intuition : les arêtes ayant un sommet commun sont connexes entre elles.
- Pour une requête : les relations ayant une variable commune sont connexes. Il décrit les connections entre les différentes relations (car critère contraignant lors d'une requête = faire correspondre les attributs)

Algorithmes (5 min.)

- Définition oreille (*hear* en anglais, vu seulement dans des docs anglophones) : arête E dont tous les sommets sont exclusivement dans E ou dans une autre arête (témoin)
- Réduction Graham-Yu-Ozsoyoglu : on applique successivement deux règles
  - on enlève les sommets isolés (on ne perd aucune information sur l'arbre, le sous-graphe engendré ne contient qu'un seul sommet donc connexe)
  - on enlève tous les arêtes inclues dans d'autres (le sous-graphe engendré par un de ses sommets relie forcément ces deux arête dans l'arbre, les relier est donc localement optimal car il suffit de rajouter une seule arête)
- Si on ne peut pas faire cela alors que l'hypergraphe est non vide, on créera nécessairement un cycle (car connexe de sommet à sommet)
- Dans l'arbre de jointure enraciné, les informations se propagent par les arêtes, des feuilles aux parents successifs
- Yannakakis : la projection se fait sur les variables utilisées par la requête et localement nécessaire pour calculer le résultat.
- Simuler exécution de Yannakakis
  - De bas en haut, supprimer les doublons induits par la sélection, puis ne garder que les clés localement partagées (enfant et parent). À la fin, ils n'y a pas de n-uplets inutiles dans la relation du haut, ils seront tous réutilisés (par l'absurde…)
  - De haut en bas, réaliser les jointures dans un tableau où un rajoute les colonnes des différentes variables. Les lignes sont "dupliquées" au fur et à mesure que les résultats s'accumulent. La taille de la jointure ne peut jamais diminuer !
- Le but est d'élager au plus tôt les résultats intermédiaires à chaque jointure. Si une variable apparaît dans un noeud et pas dans son père, il n'apparaît que dans son sous-arbre.